

HELIOS

An LLM inference engine built from scratch — and the measurements that decided its design

commit	2430d19	tests passing	269
date	25 July 2026	DST seeds green	50,000
lines of Python	15,548	scheduler bugs found by DST	15
torch	2.5.1+cpu	GPU verified on	Tesla T4 (sm_75)

The result this report is organised around. The central design bet was *quantization-asymmetric self-speculative decoding*: draft tokens cheaply from a 4-bit view of the model's own weights, then verify them in full precision. Whether that works at all rests on one number — how often the degraded draft agrees with the verifier. The project committed in advance to abandoning the feature below $\alpha = 0.60$.

measured α	0.6548	kill gate ($\alpha \geq 0.60$)	PASS
design target	0.78	target met?	NO
planned draft length γ	4	measured optimum γ	2
modelled speedup at $\gamma=8$	0.94× (a loss)	modelled at $\gamma=2$	1.39×

The feature survives its gate and misses its target. Reporting both halves is the point: the measurement changed the design — optimal draft length is 2, not 4 — and bounded the claim to roughly 1.4× modelled rather than the 2–2.4× the design had assumed. A negative-leaning result that was actually measured is worth more than an unmeasured positive one.

1 What was built

A complete LLM serving engine, written from first principles rather than assembled from libraries. The forward pass is hand-written — grouped-query attention, rotary embeddings, SwiGLU, RMSNorm — not a call into transformers, because the entire point is to own the KV cache: the model must read and write paged blocks chosen by our allocator, which no stock implementation exposes.

The seam everything follows from

The **core** — allocator, scheduler, prefix cache — is a pure state machine: no clock, no I/O, no threads, no unseeded randomness, no iteration over unordered containers. The **executor** holds all the PyTorch. They communicate only through a narrow ExecStep / ExecOutputs value type.

That seam is load-bearing in two directions. It lets a simulated executor substitute for the real one, which is what makes the entire scheduler testable under fault injection. And it is where the design's Rust scheduler and shared-memory ring buffer would attach — that component was cut, but the boundary it needs was preserved deliberately, which is the difference between a cut scope and a missing design.

Subsystem	Mechanism	Verified by
Paged KV allocator	fixed blocks, ref counts, copy-on-write	7 invariants after every op
Scheduler	iteration-level continuous batching, chunked prefill	50,000 simulation seeds
Admission	3 SLO classes, token buckets, anti-starvation aging	liveness checks per seed
Preemption	recompute vs swap, chosen by a ratio test	output identity after 14 cycles
Prefix cache	radix trie, block-aligned, LRU with pinning	output identity, hit ratio
Attention	paged; gather+SDPA and a fused Triton kernel	dense reference, 9/9 on a T4
Quantization	INT4 weights with AWQ, INT8 paged KV	error bounds derived, asserted
Speculation	int4 draft / fp verify, adaptive γ	bit-identical output
Frontend	OpenAI-compatible, incremental SSE, Prometheus	stream reassembly identity

One row is deliberately absent from that table. The 7-state KV transfer FSM (section 5) is verified by the same harness, but **nothing outside the test harness imports it** — it is a verified design, not a code path a request travels. Listing it beside paged attention would overstate it, and an earlier draft of this report did exactly that.

Why paging, stated as the trade it is

A contiguous per-sequence KV buffer sized to the maximum context length wastes memory to internal fragmentation: a 100-token request occupying a 4096-token slot wastes 97.5% of it. Paging allocates fixed-size blocks and maps them per sequence through a block table — operating-system virtual memory applied to attention state. The cost is a gather: a sequence's KV is scattered, so attention must collect it before computing scores. That gather is the tax paging pays, and eliminating it is exactly what the fused kernel in section 3 does.

2 Correctness: deterministic simulation testing

The most valuable component here is not a performance optimisation — it is the test harness, in the TigerBeetle and FoundationDB tradition. Because the scheduler is pure, an entire run is a total function of one integer seed: workload, arrival burstiness, configuration jitter, cancellations, and the fault schedule all derive from it. So the system can be driven single-threaded with a simulated clock and a fabricated executor, asserting every invariant after every step, and any failure replays exactly.

```
python -m helios.cli vopr --seed 918273 --replay
```

50,000 seeds, zero failures, 58 minutes single-threaded, re-verified after every subsequent refactor. Determinism here is not stylistic tidiness: a single `time.time()` call inside the scheduler would make a failing seed unreplayable and the harness worthless. A test asserts the module imports neither a clock nor an RNG.

What it found, and the shape of it

Fifteen real bugs. The distribution is the interesting part: **thirteen of fifteen were liveness bugs** — livelocks and starvation — not data corruption. The engine kept running, kept reporting healthy metrics, and quietly completed nothing. Conventional integration testing would have found almost none of them, because nothing crashes and no assertion trips; the system simply stops making progress, in a state that takes a specific adversarial sequence to reach.

- A **copy-on-write accounting error** that overshot the memory watermark: a shared *partially filled* tail block reported spare room while still needing a copy before it could be written. Precisely the corruption class the design document had predicted in advance.
- A **self-sustaining livelock** in which a sequence that failed admission kept its prefix-cache pin — making the very memory it was waiting for permanently unevictable.
- A sequence **admitted, preempted for memory, and re-prefilled every step forever**, because admission reserved space for the prompt but none for the tokens the request would go on to generate.
- A **leak of the entire KV pool** through a sequence that finished outside the running list, which no reaper ever visited.

The lesson that generalised: coverage is not detection

Copy-on-write turned out to be unreachable in simulation — 0 of 300 seeds exercised it, because the prefix cache shares only whole blocks, so no sequence ever wrote into a shared partial tail. Adding a fork operation produced full coverage of that path. **Reintroducing the original bug still passed 300 of 300 seeds**. The path was executed; the conditions that make it fail were not. Making forks persist did catch it — at the cost of roughly sixty spurious liveness failures per six hundred seeds, which are harness artefacts rather than engine defects. That change was reverted and the case pinned instead by a targeted test with exact control of the pool state.

The habit that came out of it is applied to every new subsystem in this report: after building a checker, deliberately reintroduce the bug it is supposed to catch, and confirm that it does.

3 Performance: single-mechanism ablations

Every performance number here is an ablation: one mechanism disabled, everything else identical, same seeded workload. For the question actually being asked — *what is this mechanism worth* — that is stronger evidence than a cross-engine comparison, because it controls for everything else by construction. It cannot answer *is this engine fast*; only a baseline can, and section 6 is explicit about that gap.

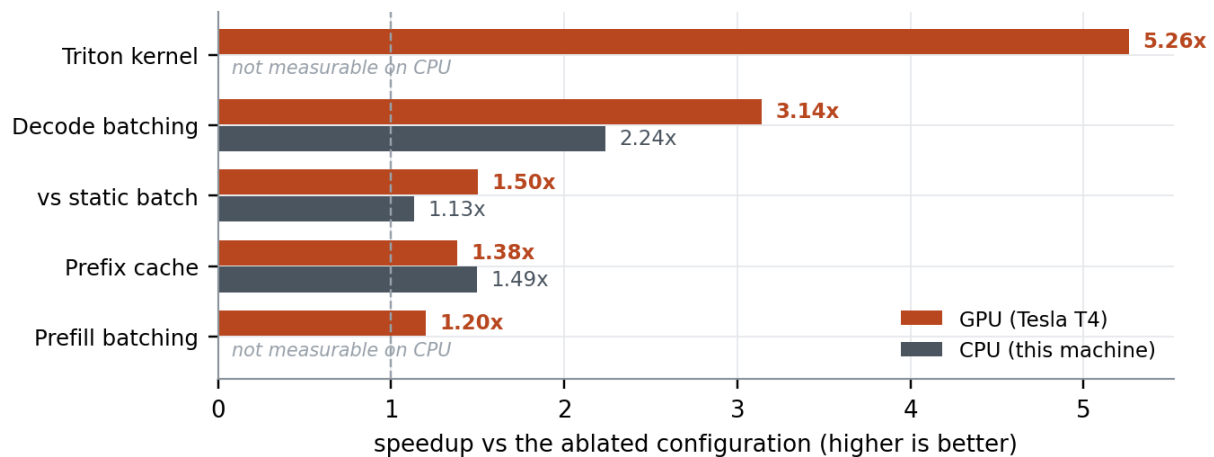


Figure 1 — Each mechanism's contribution as a single-mechanism ablation. GPU figures from a recorded Tesla T4 run; CPU figures recomputed from local artifacts at generation time.

The pattern is the one theory predicts, and it is worth stating as a prediction that held rather than as an observation. **Every mechanism pays more on the GPU than on the CPU.** Decode is memory-bandwidth bound: one query token attends over the entire context, so the step reads the whole KV cache to perform very little arithmetic. Batching amortises the weight read across sequences, and the more expensive that read is relative to the arithmetic, the more batching is worth. A GPU makes the ratio most extreme, so it is where batching pays best.

The batching trade-off is a trade, not a free win

configuration	out tok/s	TTFT p50	TPOT p50	mean batch
continuous batching	409.4	360 ms	34.1 ms	11.6
batch = 1	161.5	1884 ms	6.4 ms	1.0

Batch 1 achieves **5.4× better per-token latency** and pays **5.2× worse time-to-first-token** for it. Neither configuration is correct in the abstract: the choice is a function of whether a deployment is latency- or throughput-sensitive, which is why the engine exposes three SLO classes and admits against per-class token buckets rather than picking one policy and calling it the answer.

The fused Triton kernel

Paging costs a gather — scattered blocks collected into a contiguous tensor before scores can be computed, materialising memory the mathematics does not need. A fused kernel instead walks the block table and streams each block directly into an online-softmax accumulator, so the score matrix is never written to memory at all: FlashAttention's IO-aware idea applied to paged storage. State per program is $O(\text{head_dim})$ rather than $O(\text{context_len})$.

5.1–5.6× over the PyTorch paged path across three independent Tesla T4 runs, with 9 of 9 parity tests against that path as the correctness oracle. The kernel was written on a machine with no NVIDIA GPU and marked unverified until a real device ran the gate. Building the slow reference first was not a detour: it is the oracle a kernel must be validated against, so it was the right half to build first regardless.

4 Quantization, and the measurement that decided a design

Four-bit weight quantization was initially scoped out, on the reasoning that *quantized kernels need a GPU to be worth anything*. That reasoning conflated two separable claims, and separating them is what unlocked the most interesting result in the project.

- A **speed** claim — four-bit weights are a quarter of the bytes and decode is bandwidth-bound. This genuinely needs a GPU. On a CPU, dequantize-then-GEMM is strictly *slower* than an fp32 GEMM: the dequantization is real work and there is no bandwidth wall to relieve.
- A **numerical** claim — that four-bit weights preserve the model's output distribution well enough to be useful. This is pure arithmetic: device-independent, and fully measurable without any GPU at all.

The second claim is also the one the entire speculation design rests on. If an int4 draft does not agree with an fp verifier often enough, the scheme has no speedup available to it at *any* bandwidth. So the numerical half was built, and the kill gate became answerable on a laptop.

What was implemented

Component	Design	Measured
INT4 weights	packed nibbles, group-wise asymmetric scales	3.8× smaller than fp16
AWQ scaling	per-channel exponent search on calibration activations	1.58× lower layer error
INT8 paged KV	per-token, per-head symmetric scales	1.94× smaller than fp16
QASSD	int4 draft, fp verify, separate quantized KV shadow	$\alpha = 0.6548$

Why the scheme is safe regardless of draft quality

The property worth being able to defend cold: **speculation is correct not because the draft is good, but because the verifier decides**. An accepted token is one the target's own argmax agreed with; a rejection emits the target's argmax instead. The committed sequence is therefore the target model's greedy sequence, whatever the draft proposes. A bad draft costs throughput and nothing else.

That is asserted, not argued. A test runs identical prompts with and without the int4 draft and requires **identical token ids**. A violation would mean the engine silently serving the four-bit model's output while reporting the full-precision model's — the worst failure mode this system has, and one no smoke test would catch.

The measurement

Eight prompts spanning factual recall, arithmetic, code, and prose, because acceptance is strongly context-dependent and a single prompt family would produce a number that does not generalise. AWQ calibrated on four *disjoint* prompts — calibrating on the evaluation text is how a quantization method gets reported as better than it is.

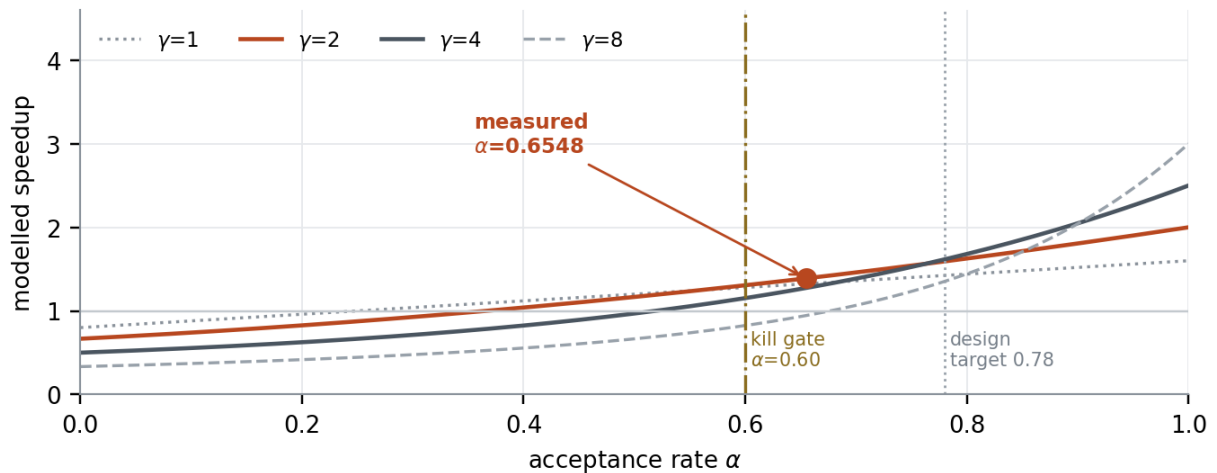


Figure 2 — Modelled speedup against acceptance rate, with the measured point, the pre-committed kill gate, and the design target marked. The optimal draft length moves with α , which is why adaptive gating is machinery worth having rather than an optimisation.

$\alpha = 0.6548$ — 220 accepted of 336 drafted, over 8 prompts and 84 verify passes. The gate passes; the 0.78 target does not. Optimal γ is 2, not the planned 4, and at $\gamma=8$ speculation becomes a net loss of 0.94× — a rejection discards every drafted token after it, so longer drafts amplify both the win and the waste.

The cost, stated in the direction that is easy to hide

Both precisions must be resident, so this is an *increase* in memory: **1.51× the target model alone**. "Four-bit quantization" sounds like a reduction, and for standalone serving it is — the int4 view by itself is 1.96× smaller. But that configuration contains no speculation. Conflating the two would be the easiest misrepresentation available here, which is why the number is reported as a ratio above 1.

Two findings that came out of the data, not the plan

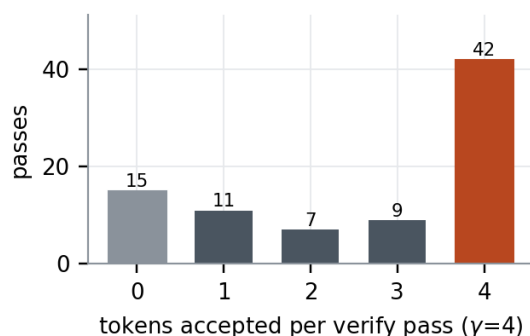


Figure 3 — Accepted-run lengths, $\gamma=4$.

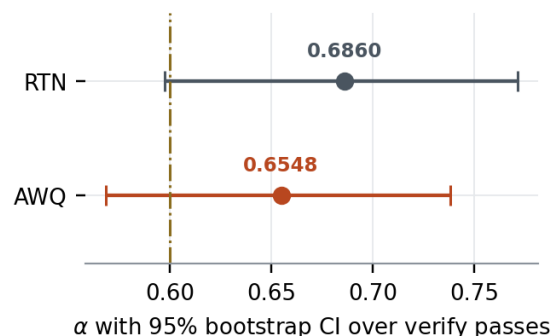


Figure 4 — AWQ vs RTN, 95% bootstrap CIs.

(a) Acceptances are correlated, and the textbook model is conservative. The run-length distribution is strongly bimodal: 42 of 84 verify passes accepted the entire draft and 15 accepted none. That falsifies the independence assumption in the standard speedup formula — a context the draft finds easy stays easy for several tokens. The consequence is favourable: measured tokens per verify pass is **3.12** against 2.55 predicted under independence, so the standard model *understates* the real value by 22%. Using the measured figure, $\gamma=4$ yields 1.56× rather than 1.27×.

(b) AWQ reduces layer error but does not improve acceptance. AWQ measures 1.58× lower per-layer output error than plain round-to-nearest, and yet α comes out at 0.6548 against RTN's 0.6860 — nominally *worse*. Before reading anything into that: the unit of independence is the verify pass, not the token, because acceptances are correlated. Treating 336 tokens as independent trials would understate the variance in exactly the direction that manufactures significance. Bootstrapping over passes and running a permutation

test gives **p = 0.64**: at this sample size the two are indistinguishable.

The likely explanation is an **objective mismatch**, and it is labelled a hypothesis because that is what it is. AWQ minimises squared output error. Acceptance depends only on whether the *argmax* survives — a far coarser property. A method can cut mean squared error substantially while flipping near-ties in the logits either way, and only the flips cost acceptance. This does not make AWQ the wrong choice for standalone four-bit serving, where output fidelity *is* the objective. It suggests the calibration target for a speculative *draft* should be top-1 agreement with the target model — which is not what AWQ optimises. A concrete, testable follow-up rather than a conclusion.

What speculation actually costs, once it is actually running

batch capped to 8	out tok/s	vs no speculation
speculation off	200.4	—
symmetric, $\gamma=2$	62.9	3.19× slower
symmetric, $\gamma=4$	65.7	3.05× slower
QASSD $\gamma=2$ (gate active)	141.7	1.41× slower

Speculation as implemented forfeits batched decode, and on a CPU that trade is never worth making. The draft loop is inherently serial and runs per sequence, so with 8 resident sequences a speculative step costs $8 \times (\gamma+1)$ unbatched forward passes where a normal step costs one batched pass. The batching win is larger than anything speculation can return. That is a property of this implementation and of CPU economics, not of the technique.

The second row worth reading is the last one. QASSD is *faster* than symmetric speculation here — not because the int4 draft is better (it is far worse: $\alpha = 0.125$ on random toy weights) but because the **adaptive gate noticed and stopped**. Measured, it disabled speculation after a single step, turning a 3.2× disaster into a 1.4× tax. Correctness comes from the verifier; throughput protection comes from the gate. Together they are what make it safe to ship a draft whose quality has not been characterised on every workload.

This ablation did not exist until an audit found that the previous one measured nothing: it ran at the default batch of 12–16, above the gate's threshold of 8, so 0 of 23 decode steps actually speculated and the row labelled "speculation" was reporting scheduling variance. Third occurrence of the same methodology error in this project, after the prefix cache and the INT8 KV pool.

The INT8 KV cache: mechanism verified, benefit not realisable here

256 KiB KV pool	mean decode batch	TTFT p50	out tok/s
fp32 KV	4.47	2201 ms	73.8
INT8 KV	10.31	1356 ms	54.0

The mechanism does exactly what it claims: **2.31× more resident sequences** at the same byte budget, and **1.62× better time-to-first-token**, because sequences are admitted sooner. And throughput is **27% worse**, because on a CPU the dequantization is real work with no bandwidth bottleneck to offset it. On a GPU, where reading the KV cache *is* the decode bottleneck, halving those bytes should more than pay for the dequantization — a prediction from the same roofline argument that correctly anticipated the GPU/CPU pattern in Figure 1, and *not* claimed as a result.

Getting this measurement right took two attempts, and the first was wrong in an instructive way. Run in a 4 MiB pool, both configurations kept every sequence resident — mean batch 14.95 in both — so the ablation measured nothing but dequantization overhead, and INT8 looked 20% slower. That is a mechanism evaluated outside the regime it exists for. The pool had to be made genuinely binding before the comparison meant anything at all.

5 Distributed KV migration as a state machine

Prefill and decode have opposite bottlenecks. Prefill has many query tokens over a short context: high arithmetic intensity, compute-bound. Decode has one query token over the whole context: almost no arithmetic per byte read, bandwidth-bound. Interleaved on one device they make each other worse — a single long prompt adds its full duration to every resident sequence's inter-token latency. Chunked prefill bounds that, and bounding is not eliminating.

Disaggregation removes the contention instead, by putting the two on separate devices and transferring the prompt's KV cache between them.

What follows is a verified design, not a shipped feature, and the distinction is worth making precisely. Nothing outside the test harness imports this module — the engine, the scheduler and the runner never call it. With one device there is no transfer to perform, because the blocks are already where they need to be; routing them through a transfer FSM would be theatre, a code path existing to make a claim true rather than to do work. What is real is the protocol: a state machine with partial-failure semantics worked out and mechanically checked before any transport exists. That is worth more than transport code with the semantics left implicit — but it is not something a user can switch on, and an earlier draft of this report listed it as though it were.

Why a state machine and not a coroutine

The design deliberately separates SENT (the sender believes it is done) from RECEIVED (the receiver confirms). Every interesting fault lives in the gap between those two, and an `async/await` formulation collapses them by default — which makes "sender finished, receiver never acknowledged" *unrepresentable*, and therefore untestable. Seven explicit states with a legal-transition table; the table is the specification, so an unlisted edge raises rather than being absorbed.

Five fault kinds are modelled — receiver out of memory, link timeout, checksum mismatch, lost acknowledgement, mid-flight abort. Each returns blocks from **both** partitions, because handling one side leaks the other, and a leak stays invisible until the pool is exhausted — at which point it looks like a capacity problem rather than a bug. Two rules fall directly out of the ownership analysis: the sender may not release until the receiver confirms, or a lost acknowledgement becomes unrecoverable rather than merely slow; and a client abort after the bytes are on the wire must resolve as a *failure* rather than an abort, so that ownership is settled before anything is freed.

Verified by detection, not by coverage

Half of all simulation seeds run the FSM alongside the scheduler under randomised faults, asserting six FSM invariants plus two pool invariants the FSM cannot check itself. Coverage across 200 seeds reached every fault kind. Then — applying the lesson from section 2 — the classic partial-failure bug was reintroduced deliberately: `fail()` returning the sender's blocks but not the receiver's. **Caught by 31 of 60 seeds**, with a message naming the exact accounting discrepancy. Making an abort legal after SENT was caught by 15 of 60.

An error this produced, worth recording

Whether to transfer a prompt's KV or simply recompute it is a decision that flips with the hardware, so it is computed from bandwidth rather than assumed. The first version of that arithmetic put a 2000-token KV cache at 2.6 GB when it is **262 MB** — a factor of ten, which inverted the verdict for PCIe from "wins by 11x" to "roughly break-even". It was caught by a test written against the real formula, not by re-reading the comment. Corrected, the conclusion is sharp: **within a node, transferring is clearly right; across a commodity network it is not.** The crossover sits at the machine boundary. A design that always transferred would be strictly worse than no disaggregation at all on the wrong side of that line.

6 What a deliberate audit found

Everything above was written, tested, and committed before this section existed: 263 tests green, 5,000 simulation seeds green. The audit was run anyway, on the premise that **a green suite is evidence the code does what the tests say, not that the claims are true**. Four of the five findings were claim errors rather than code errors, and no test was failing for any of them.

Found	Why it mattered
QuantLinear memoised the dequantized fp weight, so after one forward pass an int4 layer held 26% MORE memory than the fp16 layer it replaced — while stored_bytes() reported a 3.8x saving	Critical: every quantization memory claim in the project was false in practice. Caching is now opt-in and off by default, and resident_bytes() exists so the packed size and the real size cannot be conflated again.
The speculation ablation ran at a batch above the adaptive gate, so 0 of 23 decode steps actually speculated	High: a row labelled "speculation" was measuring scheduling variance. A dedicated suite now caps the batch so the mechanism is on the critical path.
memory_overhead() reported only weights, omitting the entire second KV cache that QASSD allocates	Medium: the quoted number was true and incomplete — harder to catch than a false one, because nothing about it looks wrong.
The KV transfer FSM was listed as "Built" beside paged attention, when nothing outside the test harness imports it	Medium: a verified design was presented as a shipped feature. Relabelled, with the grep that proves it.
A benchmark run was contaminated by a concurrent job on the same machine	Low: it inverted the $\gamma=2$ / $\gamma=4$ ordering. Re-run idle; the inversion disappeared.

Each fix carries a regression test, and each of those tests was **mutation-checked**: the bug it exists to catch was deliberately reintroduced and the test confirmed to fail. One result from that exercise is worth more than the others. When the accept logic was mutated to commit the draft's token instead of the verifier's, **every symmetric-speculation parity test passed** — because at $\alpha = 1.0$ the draft token and the verified token are identical, so the bug is invisible. Only the asymmetric test failed.

That is the general lesson, and it is the same one section 2 records about copy-on-write coverage, arriving from a different direction: **a test suite can be large and green and still have no coverage of the thing that matters**. The only reliable check is to break the code on purpose and watch what fails.

7 What is not built

This section exists because the project's governing rule was that a claim must never outrun its evidence — and the only way to keep that rule is to write the boundary down rather than leave a reader to discover it.

Not built	Why, precisely
A fused int4 GEMM	The quantization is numerically real and verified; the SPEEDUP is not. QuantLinear dequantizes and then calls a normal GEMM, so on a CPU it is slower than fp32. Every quantization speedup in this report is labelled modelled.
vLLM comparison	The harness is written with matched controls — same model directory, dtype, KV byte budget, seeded arrival trace, discarded warm-up — and has never been run, because vLLM requires CUDA. No comparison is claimed.
W4A4 (4-bit activations)	The available GPU has no fast int4 activation path, so a W4A4 draft would not be cheaper than the verify and the premise fails. A hardware constraint rather than a gap in the work.
Real multi-device disaggregation	The FSM, the block accounting, and the transfer-versus-recompute decision are built and verified. The transport is simulated: there is no second device to move bytes to.
Rust frontend and scheduler	Cut from scope and still cut. The ExecStep seam it would attach to was preserved deliberately, which was the point.
α on a larger model	The 0.5B result is marginal. Quantization error at a fixed bit-width falls with model size, so a 7B model should do better — a hypothesis, untested, and not claimed.

One caveat applies to every CPU figure in this report: the local benchmark model is a small random-weight toy, so absolute tokens per second are meaningless. The *ratios between ablations* are the result. What makes the correctness claim credible instead is coherent text from a real pretrained checkpoint — Qwen2.5-0.5B on a T4 in fp16 — because paging, block tables, rotary embeddings, GQA head expansion, normalisation, batched prefill, batched decode, fp16 numerics, and sampling must all be simultaneously correct for a pretrained model to produce sensible output. A toy model with random weights can only ever demonstrate self-consistency: an error shared by the implementation and its own reference would pass every such test.

8 What the process taught

Three bugs were structurally invisible without real hardware

The GPU sessions found four bugs, and **three of them could not have been caught on the development machine at all** — while 178 tests were passing. Qwen2 attention biases were silently dropped, because the loader skipped checkpoint tensors it had no slot for and the toy model is written by the same code that reads it. Model weights were never moved to the device, because `.to(dtype)` is not `.to(device)` and the two coincide when there is only one device. And the executor converted the resulting exception into a retryable fault, turning a hard failure into an infinite retry loop whose only symptom was an empty result list.

The regression tests written afterwards assert the *invariant* — every parameter on the device the engine builds inputs on — rather than the symptom, because the symptom is unobservable where the test runs.

A documented conclusion that was simply wrong

For a period this project recorded, as a finding, that "continuous batching needs a GPU to pay" — because the measured batching win was zero. A matmul microbenchmark then showed roughly 10× from batching GEMMs on a CPU at 32 sequences. The engine had been running one forward pass per sequence: a missing optimisation in the executor, not a property of the hardware. Fixing it required no GPU and produced 2.24×. The lesson that stuck: a negative result about *hardware* deserves more suspicion than a negative result about code, because it is unfalsifiable from where you happen to be standing.

Measuring a mechanism outside its regime reads as a regression

This happened three times — the prefix cache on a 32-token shared prefix, prefill batching hidden behind long generations, and the INT8 KV cache in a pool that was not binding. Each looked worthless or actively harmful until the workload was constructed so the mechanism had something to do. Dedicated suites now exist for each. The general form: an optimisation's benchmark has to be designed alongside the optimisation, or the default workload silently decides the verdict.

Pre-committing to a kill threshold changed the outcome

The 0.60 acceptance floor was written down before anything was built, and the measurement came in at 0.655 — close enough that a threshold chosen afterwards would have been indefensible in either direction. Because it was fixed in advance, the result is a decision rather than a rationalisation: the feature ships, its optimal parameter is not what was planned, and its expected benefit is bounded by a number instead of by hope.

Appendix Reproduction and provenance

```
# full test suite
PYTHONPATH=python python -m pytest tests -q

# deterministic simulation harness; any failure replays from its seed
PYTHONPATH=python python -m helios.cli vopr --seeds 50000
PYTHONPATH=python python -m helios.cli vopr --seed 918273 --replay

# ablations -> JSON artifacts -> generated docs/BENCHMARKS.md
python bench/loadgen.py --model artifacts/bench_model --suite all
python bench/report.py

# the acceptance-rate measurement and its significance analysis
python bench/measure_alpha.py --model artifacts/qwen05b --gamma 4
python bench/measure_alpha.py --model artifacts/qwen05b --no-awq \
    --out artifacts/alpha_rtn.json
python bench/alpha_significance.py

# GPU gates: device report, suite, Triton parity, real model end-to-end
python bench/kaggle_gpu_run.py

# this report
python bench/make_report.py --out docs/HELIOS-REPORT.pdf
```

Field	Value
commit	2430d19
generated	25 July 2026
python / torch	3.12.0 / 2.5.1+cpu
platform	Windows-11-10.0.26200-SP0
lines of Python (src, tests, bench)	15,548
tests passing	269
simulation sweep	50,000 seeds, 0 failures
sweep including the transfer FSM	5,000 seeds, 0 failures
α measurement	qwen05b, $\gamma=4$, group=128, awq=True

Every figure and table above is generated from a JSON artifact in `artifacts/` by `bench/make_report.py`. No number in this document is typed in by hand, for the same reason `docs/BENCHMARKS.md` is generated rather than written: a report that can be edited independently of its measurements will eventually disagree with them, and nobody will notice.

Artifacts absent at generation time

Listed rather than silently omitted, so a report built from a partial run says so on its face: `gpu_run.json`, `vllm_baseline.json`.